03

OOP Fundamentals

This chapter introduces OOP, a popular programming paradigm that organizes code and data into objects. These objects interact to model real-world problems, making software more adaptable, easier to maintain, and simpler to understand.

Why Learn OOP?

Understanding OOP fundamentals, including core principles such as encapsulation and advanced design guidelines like SOLID, is crucial for building robust and scalable software. These principles provide a structured approach to managing complexity, reducing code redundancy, and ensuring that code can be reused across various projects, while also improving its interoperability and future adaptability.

To work toward these objectives, OOP relies on three fundamental pillars and technical components. Understanding OOP concepts and SOLID principles is essential for writing clean, maintainable, and scalable code. Mastery of these principles enables developers to create flexible and robust applications and getting ready for cutting-edge topics, OOP enables the backbone of general software development. To deepen your expertise, explore advanced topics like design patterns, concurrency, and performance optimization.

Cornerstones of Object-Oriented Programming

Object-oriented programming is built on four fundamental pillars: encapsulation, abstraction, inheritance, and Polymorphism.

These principles guide how we organize code and design software. The other techniques and design patterns stem from these foundations, and they are important for understanding software design. We will cover these principles with practical examples.

Encapsulation

Encapsulation is the concept of bundling data as attributes and logic as methods, and then putting related attributes and methods together within a single unit, typically a class. This approach helps manage complexity by hiding internal details and exposing only necessary interfaces, which enhances code modularity and security. A classic example of an object is a bank, where a specific bank account is called an instance.

Let's demonstrate encapsulation in Java with a simple class representing a "Person":

```
public class Person {
    // Private data members (attributes)
    private int age;
    private boolean isStudent;

    // Public constructor
    public Person(int age, boolean isStudent) {
        this.age = age;
        this.isStudent = isStudent;
    }

    // Public getter methods (accessors)
    public int getAge() {
        return age;
    }

    public boolean isStudent() {
        return isStudent;
    }

    // Public setter methods (mutators)
    public void setAge(int age) {
        this.age = age;
    }

    public void setStudent(boolean isStudent) {
        this.isStudent = isStudent;
    }
}
```

- The Person class has private attributes (age and isStudent) that cannot be accessed directly from outside the class.
- Public getter methods (getAge() and getStudent), allow external code to read (access) the private attributes.
- Public setter methods, allow external code to modify (update) the private attributes, while ensuring data integrity.

This implementation ensures that the Person class's internal state is protected, and external code interacts with it only through controlled methods.

When to use encapsulation?

Encapsulation is a good practice in the following scenarios:

- **Protecting data integrity:** When you need to ensure an object's data remains consistent and valid. For example, in the Person class, setting a negative age would be prevented by the setAge() method, which can include validation logic to reject invalid values.
- **Controlling access and improving security:** Encapsulation restricts direct access to sensitive data. While attributes like isStudent might be accessed through specific methods, encapsulation prevents unauthorized modifications from other parts of the system, which is crucial for maintaining data security and preventing unintended side effects.
- **Simplifying complex systems:** When designing classes that can be reused across different applications, the Person class example demonstrates how encapsulation and accessors contribute to a more manageable and secure interaction pattern.

Common pitfalls

While encapsulation is powerful, avoid these common mistakes:

- **Overuse of getters and setters:** Creating getter and setter methods for every attribute can create code verbosity and hinder maintenance.
- **Under encapsulation:** Failing to hide internal details can lead to tight coupling and reduced modularity. For instance, if some Person attributes are made public, changes to these attributes could ripple through other parts of the system.

Abstraction

Abstraction can simplify complex systems by hiding unnecessary details. It separates the "what" (an object does from the "how" (it does it), allowing developers to work with objects at a higher level. For example, when you drive a car, you interact with the steering wheel and pedals, without needing to understand the underlying engine mechanics. This concept commonly is implemented using abstract classes and interfaces.

To see how abstraction works in practice, let's create a Shape class and a Drawable interface, which define geometric properties and drawing capabilities.

When to use abstraction?

Abstraction is particularly useful in scenarios like:

- **Managing complex interactions:** Abstraction helps provide a clear and consistent interface for complex functionality, for instance, a Shape interface might define methods for calculating area and perimeter, ensuring that all shapes can be interacted with in a standardized way.
- **Promoting code flexibility:** When you anticipate that subclasses will provide specific implementations of abstract methods, abstraction allows you to define a common interface while leaving the details of those methods to the subclasses, enhancing extensibility and supporting different shape-specific behaviors.
- **Supporting extensibility:** Abstraction makes it easier to extend systems without modifying existing code. For instance, adding a new shape, like a triangle, to the Shape hierarchy only requires implementing area(), ensuring compatibility with existing drawing functionalities.

Abstraction vs. Encapsulation

Abstraction and encapsulation are distinct but complementary OOP principles, often confused because both involve hiding details. Here's how they differ:

Characteristics	Abstraction	Encapsulation
Focus	Simplifies complexity by exposing only what an object does, hiding the underlying implementation details. It's about what to do.	Bundles data and methods into a single unit (a class) and protecting data by restricting direct access.
Purpose	Simplifies user interaction and promotes flexibility by defining high-level behaviors.	Controls data integrity and maintainability by restricting access to an object's data.
Implementation	Uses abstract classes and interfaces, e.g., the Shape interface defines methods like area() and perimeter() without details.	Uses access modifiers (e.g., private, public) and getter/setter methods to control data access. For instance, the Person class keeps age private and provides controlled access via getAge() and setAge().

By understanding these differences, you can apply abstraction to simplify interfaces and encapsulation to protect data, ensuring robust and user-friendly systems.

Inheritance vs. Composition

Inheritance is a mechanism where one class (a subclass or derived class) inherits properties and behaviors from another class (a superclass or parent class), promoting code reuse and creating hierarchical relationships between classes. Think of inheritance like a family tree, where children inherit traits from their parents, and grandchildren inherit traits from both their parents and grandparents. The concepts are covered and exemplified by the implementation of the Shape and Drawable classes.

Common patterns of class hierarchy

Building on the concept of inheritance, here are common patterns for structuring class hierarchies in Java:

Single Inheritance

A subclass extends only one superclass. This is the standard type of inheritance supported in Java. Below is an example of single inheritance.

Single Inheritance

In the above example, we demonstrate single inheritance by creating a dog class that extends the Animal class. The Dog class inherits the eat() method from Animal.

Multilevel Inheritance

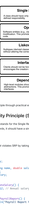
A subclass then inherits from another subclass, creating a chain of inheritance, is called multilevel inheritance. Consider a scenario with three classes: Animal, Mammal, and Dog, where Mammal extends Animal, and Mammal is the superclass of Dog.

Multilevel Inheritance

In the above example, the Dog class inherits behaviors from the Animal and Mammal classes. Multilevel inheritance is useful when you have classes that exhibit a hierarchical relationship, with each subclass specializing and adding new behaviors to the existing hierarchy.

Hierarchical Inheritance

In hierarchical inheritance, multiple subclasses inherit from a single superclass, forming a hierarchical structure. In the example below, Dog and Cat are two subclasses, Cat specifically inherits from the Animal class, forming a hierarchical relationship.



Hierarchical Inheritance

Hierarchical inheritance is beneficial when multiple classes share common attributes or behaviors from a single superclass.

When to use inheritance?

Inheritance is particularly useful in the following scenarios:

- When subclasses share common attributes or behaviors, as in our Shape example.
- When multiple classes share common attributes or methods, a superclass can define them once, allowing all subclasses to inherit and reuse the shared code.
- When creating specialized versions of a base class, such as adding a Triangle subclass to the Shape hierarchy.

Drawbacks of inheritance

While inheritance promotes code reuse, it introduces certain challenges. Here are the key drawbacks to consider:

- **Tight coupling:** Inheritance creates a strong dependency between the superclass and subclasses. Changes to the superclass may inadvertently affect all subclasses, potentially leading to unintended consequences. For example, if we add a new method to the Shape class, all subclasses must now implement this method, even if it doesn't apply. For example, adding a method like calculatePerimeter() might be meaningless for a Triangle if its sides aren't equal. This tight coupling can reduce flexibility and make the system harder to maintain and extend.
- **Inflexible hierarchies:** Once a class is extended, changes to its parent class can have far-reaching effects on all its subclasses, which can sometimes hinder the addition of new features or modifications to the existing hierarchy.
- **Complexity in multiple inheritance:** While not supported in Java, multiple inheritance (where a class can inherit from more than one superclass) can lead to complex dependencies and conflicts, commonly known as the "diamond problem." Java avoids this by allowing inheritance from only one superclass but supports multiple interfaces to mitigate this issue.

When to use composition?

Composition is a design principle where complex objects are built by combining simpler, independent components rather than inheriting from a parent class. It promotes flexibility by defining relationships between objects as "has-a" rather than "is-a" relationships, allowing for more modular and maintainable code.

Consider a media player as a practical example. Different types of media, such as audio, video, and streaming content, require different playback logic. Instead of creating a complex inheritance hierarchy, we can define a MediaPlayer interface with a play() method and then create separate classes like AudioPlayer, VideoPlayer, and StreamingPlayer that implement this interface. This approach ensures flexibility and avoids the rigidity and complexity that inheritance might introduce.

Types of Polymorphism

Polymorphism is fundamental in object-oriented programming, typically categorized into two main types: compile-time (static) and runtime (dynamic) polymorphism.

Compile-Time polymorphism via method overloading

Method overloading allows a class to have multiple methods with the same name but different parameters. The compiler determines which method to call based on the number, type, and order of the arguments provided at compile time. This approach enhances code readability and flexibility, providing a clear and intuitive way to handle similar operations with different parameters.

Here's an example of method overloading in Java:

```
class MathOperations {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public String add(String str1, String str2) {
        return str1 + str2;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations mathOp = new MathOperations();

        int sum = mathOp.add(5, 10);
        double doubleSum = mathOp.add(3.5, 2.5);
        String concat = mathOp.add("Hello, ", "World");

        System.out.println("Sum of integers: " + sum);
        System.out.println("Sum of doubles: " + doubleSum);
        System.out.println("Concatenated string: " + concat);
    }
}
```

In this example:

- The MathOperations class defines multiple add() methods with different parameter lists.
- The compiler selects the appropriate add() method based on the arguments passed, enhancing code readability by using a consistent method name for related operations.

Runtime polymorphism via method overriding

Method overriding occurs when a subclass provides a specific implementation for a method already defined in its superclass. The method to be executed is determined at runtime based on the actual type of the object, not the reference type. This is often referred to as dynamic dispatch, a hallmark of runtime polymorphism.

Here's an example of method overriding in Java:

```
class Animal {
    public void eat() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    public void eat() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    public void eat() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        Animal myDog = new Dog();
        Animal myCat = new Cat();

        myAnimal.eat();
        myDog.eat();
        myCat.eat();
    }
}
```

In this example:

- The Animal class defines a generic sound method.
- The Dog and Cat subclasses override the sound method with specific implementations.
- At runtime, the JVM determines the actual type of the object (Dog or Cat) and calls the appropriate sound method, even though the reference type is Animal.

This demonstrates polymorphism's flexibility, allowing for more adaptable and extensible code.

When to use polymorphism?

Polymorphism is a powerful technique in object-oriented programming that offers several key benefits:

- **Simplifying complex systems:** It allows for a unified interface with different underlying implementations, simplifying the interaction with various object types.
- **Enhancing code reusability:** Polymorphism promotes code reuse by enabling the creation of flexible, reusable methods that can work with different object types.
- **Improving maintenance:** The ability to add new subclasses without modifying existing code helps in maintaining a clean and extensible codebase.
- **Supporting dynamic behavior:** Runtime polymorphism allows for flexible method calls that can adapt to different object types during execution.

SOLID Principles

SOLID is a set of five design principles that aim to make software more understandable, flexible, and maintainable. These principles, introduced by Robert C. Martin, are fundamental to object-oriented programming and design. Let's explore each principle with examples and discussions on their benefits.

Single Responsibility Principle (SRP)

The SRP states that a class should have only one reason to change, meaning it should focus on a single task or responsibility. Consider a report management system where we initially have a single class handling report generation, saving, and printing:

```
// Report.java
public class Report {
    private String content;

    public Report(String content) {
        this.content = content;
    }

    public void generateReport() {
        // Logic to generate the report
        System.out.println("Generating report...");
    }

    public void saveToFile(String filename) {
        // Logic to save the report to a file
        System.out.println("Saving report to " + filename + ".txt");
    }

    public void printReport() {
        // Logic to print the report
        System.out.println("Printing report...");
    }
}
```

The Report class mixes three distinct responsibilities: generating reports, saving reports, and printing reports. This violates SRP because changes to any of these functionalities would require modifying the class.

To improve this design, we can separate these responsibilities into three classes:

```
// ReportGenerator.java
public class ReportGenerator {
    private Report report;

    public ReportGenerator(Report report) {
        this.report = report;
    }

    public void generateReport() {
        // Logic to generate the report
        System.out.println("Generating report...");
    }
}

// ReportSaver.java
public class ReportSaver {
    public void saveToFile(Report report, String filename) {
        // Logic to save the report to a file
        System.out.println("Saving report to " + filename + ".txt");
    }
}

// ReportPrinter.java
public class ReportPrinter {
    public void printReport(Report report) {
        // Logic to print the report
        System.out.println("Printing report...");
    }
}
```

This refactored design ensures each class has a single responsibility:

- ReportGenerator handles report generation.
- ReportSaver manages saving reports to files.
- ReportPrinter deals with printing reports.

This separation enhances maintainability and flexibility. Changes to one aspect (e.g., adding new saving formats) don't affect other functionalities, making the system more robust and easier to extend.

Open/Closed Principle (OCP)

The OCP states that software entities should be open for extension but closed for modification. This principle encourages adding new functionality without altering existing code, typically achieved through abstraction and polymorphism.

Consider a payment processing system initially supporting only credit card payments:

```
// CreditCardPayment.java
public class CreditCardPayment {
    public void processPayment(double amount) {
        // Logic to process credit card payment
        System.out.println("Processing credit card payment of $" + amount);
    }
}
```

To add support for PayPal payments without modifying the existing code, we can introduce an interface and implement it in new classes:

```
// PaymentProcessor.java
interface PaymentProcessor {
    void processPayment(double amount);
}

// CreditCardPayment.java
public class CreditCardPayment implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        // Logic to process credit card payment
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// PayPalPayment.java
public class PayPalPayment implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        // Logic to process PayPal payment
        System.out.println("Processing PayPal payment of $" + amount);
    }
}
```

This approach allows adding new payment methods by creating new classes that implement the PaymentProcessor interface, without modifying existing code.

Liskov Substitution Principle (LSP)

The LSP states that subclasses should be substitutable for their parent classes without affecting the correctness of the program. Violations of this principle often indicate that inheritance is not the appropriate relationship.

Consider a bird hierarchy where not all birds can fly:

```
// Bird.java
public class Bird {
    public void fly() {
        // Default flying behavior
        System.out.println("Bird is flying");
    }
}

// Penguin.java
public class Penguin extends Bird {
    @Override
    public void fly() {
        // Penguins can't fly, throw an exception
        throw new UnsupportedOperationException("Penguins can't fly");
    }
}
```

This violates LSP because penguins can't fly, but they're still birds. To fix this, we can introduce a more specific hierarchy:

```
// Bird.java
interface Bird {
    void eat();
    void sleep();
}

// FlyingBird.java
interface FlyingBird extends Bird {
    void fly();
}

// Penguin.java
class Penguin implements Bird {
    @Override
    public void eat() {
        System.out.println("Penguin eating");
    }

    @Override
    public void sleep() {
        System.out.println("Penguin sleeping");
    }
}

// Eagle.java
class Eagle implements FlyingBird {
    @Override
    public void eat() {
        System.out.println("Eagle eating");
    }

    @Override
    public void sleep() {
        System.out.println("Eagle sleeping");
    }

    @Override
    public void fly() {
        System.out.println("Eagle flying");
    }
}
```

This revised design ensures that all birds can eat and sleep, but only specific birds can fly, maintaining the correctness of the hierarchy.

Interface Segregation Principle (ISP)

The ISP states that clients should not be forced to depend on interfaces they don't use. This principle helps prevent bloated interfaces and promotes more focused, cohesive interfaces.

Consider a multi-function printer interface:

```
// MultiFunctionPrinter.java
interface MultiFunctionPrinter {
    void print(Document d);
    void scan(Document d);
    void fax(Document d);
    void staple(Document d);
}
```

This interface forces implementing classes to deal with all these functions, even if they don't need them. We can segregate the interface into smaller, more specific interfaces:

```
// Printer.java
interface Printer {
    void print(Document d);
}

// Scanner.java
interface Scanner {
    void scan(Document d);
}

// Fax.java
interface Fax {
    void fax(Document d);
}

// Stapler.java
interface Stapler {
    void staple(Document d);
}

// SimplePrinter.java
class SimplePrinter implements Printer {
    @Override
    public void print(Document d) {
        // Implementation for simple printer
        System.out.println("Printing document");
    }
}

// MultiFunctionPrinter.java
class MultiFunctionPrinter implements Printer, Scanner, Fax, Stapler {
    @Override
    public void print(Document d) {
        // Implementation for multi-function printer
        System.out.println("Printing document");
    }

    @Override
    public void scan(Document d) {
        // Implementation for multi-function printer
        System.out.println("Scanning document");
    }

    @Override
    public void fax(Document d) {
        // Implementation for multi-function printer
        System.out.println("Faxing document");
    }

    @Override
    public void staple(Document d) {
        // Implementation for multi-function printer
        System.out.println("Stapling document");
    }
}
```

This segregated design allows implementing classes to only deal with the functions they need, improving flexibility and reducing unnecessary dependencies.

Dependency Inversion Principle (DIP)

The DIP states that high-level modules should not depend on low-level modules, both should depend on abstractions. This principle promotes loose coupling between different parts of the system.

Consider a simple notification system:

```
// EmailNotification.java
class EmailNotification {
    public void sendNotification(String message) {
        // Logic to send email notification
        System.out.println("Sending email: " + message);
    }
}

// NotificationService.java
class NotificationService {
    private EmailNotification emailNotification;

    public NotificationService() {
        this.emailNotification = new EmailNotification();
    }

    public void notifyUser(String message) {
        emailNotification.sendNotification(message);
    }
}
```

This design tightly couples the NotificationService to the EmailNotification class. To invert the dependency, we can introduce an abstraction:

```
// MessageSender.java
interface MessageSender {
    void sendMessage(String message);
}

// EmailNotification.java
class EmailNotification implements MessageSender {
    @Override
    public void sendMessage(String message) {
        // Logic to send email notification
        System.out.println("Sending email: " + message);
    }
}

// NotificationService.java
class NotificationService {
    private MessageSender messageSender;

    public NotificationService(MessageSender ms) {
        this.messageSender = ms;
    }

    public void notifyUser(String message) {
        messageSender.sendMessage(message);
    }
}
```

This revised design allows easy substitution of different notification methods without modifying the NotificationService class.

Best practices

Adhering to best practices ensures that the implementation of these principles results in robust, maintainable, and scalable software. Here's how to apply these principles effectively:

Single Responsibility Principle (SRP)

- **Identify distinct responsibilities:** Clearly define each class's purpose and ensure it handles only one aspect of the system's functionality.
- **Use composition:** Instead of creating large classes with multiple responsibilities, use composition to combine smaller, focused classes.
- **Regular refactoring:** As requirements change, regularly review and refactor your code to maintain adherence to SRP.

Open/Closed Principle (OCP)

- **Use abstract classes and interfaces:** Define stable interfaces that can be extended without modifying existing code.
- **Implement new features through extension:** When adding new functionality, create new classes that implement existing interfaces rather than modifying existing classes.
- **Avoid modifying existing code:** Minimize changes to tested and stable code to reduce the risk of introducing bugs.

Liskov Substitution Principle (LSP)

- **Ensure proper inheritance hierarchies:** Only use inheritance when there's a true "is-a" relationship.
- **Override methods carefully:** When overriding methods, ensure the behavior remains consistent with the parent class.
- **Use composition when appropriate:** If inheritance doesn't fit well, consider using composition instead.

Interface Segregation Principle (ISP)

- **Keep interfaces small and focused:** Create specific interfaces for different functionalities.
- **Avoid "fat" interfaces:** Don't force classes to implement methods they don't need.
- **Use multiple interfaces:** Allow classes to implement multiple interfaces if they need different functionalities.

Dependency Inversion Principle (DIP)

- **Depend on abstractions:** Use interfaces or abstract classes to define dependencies.
- **Use dependency injection:** Provide dependencies through constructors or setters rather than creating them within the class.
- **Avoid direct instantiation:** Use factory patterns or dependency injection frameworks to manage object creation.

Common pitfalls and tips

- **Overuse of design patterns:** While these principles are important, avoid forcing complex patterns where simpler solutions would suffice.
- **Balance between SOLID and other principles:** Remember that SOLID is just one set of principles. Other principles like DRY (Don't Repeat Yourself) and KISS (Keep It Simple, Stupid) are also important.
- **Regular code reviews:** Implement code reviews to ensure adherence to these principles and catch violations early.
- **Continuous learning:** The software development landscape evolves rapidly. Stay updated with new patterns and principles.

Remember, these principles are guidelines rather than strict rules. The goal is to create maintainable, flexible, and understandable code that serves the project's needs effectively.

To adhere to the SRP effectively, consider the following guidelines:

- Aim to define a clear role for each class, focusing on one specific task.
- If a class handles multiple tasks, refactor it into smaller, focused classes with single responsibilities.
- Design classes so that changes to one task don't impact others.

Open/Closed Principle (OCP)

The Open/Closed Principle (OCP) states that software entities, such as classes, should be open for extension but closed for modification. This means you can add new functionality without altering existing code.

Violation of OCP

Here's an example of a class that violates OCP by requiring changes to support new shapes:

```
class Rectangle {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double calculateArea() {
        return width * height;
    }
}

class AreaCalculator {
    public double calculateArea(Rectangle rectangle) {
        return rectangle.calculateArea();
    }
}
```

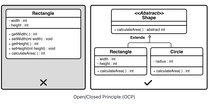
In the above example, the AreaCalculator class works only with Rectangle objects. Adding support for new shapes, like circles or triangles, would require modifying its code. This makes the system harder to maintain and prone to errors, as each new shape requires altering the core logic.

Fixing the violation

- We can refactor this design to support new shapes without changes by introducing an abstract Shape class that defines a common behavior for all shapes: calculating their area.
- Specific shapes, like Rectangle and Circle, inherit from Shape and provide their area calculations.

This design allows new shapes, such as triangles, to be added by creating new classes that inherit from Shape, without modifying AreaCalculator or existing shape classes.

Below is the visual representation of the classes and implementation of OCP:



Open/Closed Principle (OCP)

Best practices

With this extensible design in place, here are guidelines to ensure OCP compliance:

- Consider introducing abstract classes or interfaces to create flexible blueprints that classes can extend with new functionality.
- Allow subclasses to override methods to provide specific behaviors, such as unique area calculations for different shapes.
- Use polymorphism to treat objects of different classes, like various shapes, uniformly through a common interface or base class.

Liskov Substitution Principle (LSP)

The Liskov Substitution Principle (LSP) states that objects of a derived class should be able to replace objects of the base class without affecting the correctness of the program. In other words, if class A is a subtype of class B, then instances of class B should be replaceable with instances of class A without causing issues.

Violation of LSP

Here's an example of a design that violates LSP by assuming all birds can fly:

```
class Bird {
    public void fly() {
        System.out.println("Flying in the sky.");
    }
}

class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Ostriches cannot fly.");
    }
}

// Program calls Bird.fly() to test bird behavior
```

In this code, the Ostrich class inherits from Bird but throws an exception for fly, as ostriches cannot fly. This breaks the expectation that any Bird can fly when a program tests this behavior. This violates the principle that derived classes should behave as expected when replacing their base class, making the design unreliable.

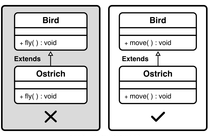
Fixing the violation

To align with LSP, we can refactor this hierarchy to ensure substitutability:

- Instead of assuming all birds can fly, we redefine the Bird class as an abstract class with a more general behavior, such as moving, that all birds can perform.
- The Bird class defines a move method, which each bird implements according to its abilities. For example, a Sparrow might implement move by describing flying in the sky, while an Ostrich implements move by describing running on land.

This design ensures that any derived class, like Ostrich, can replace Bird seamlessly, maintaining the program's correctness.

Here is the visual representation of the classes and implementation of LSP:



Liskov Substitution Principle (LSP)

Best practices

With this flexible design in place, here are the guidelines to ensure LSP compliance:

- Ensure that derived classes maintain the behavioral compatibility of their base classes. Methods in derived classes should follow the same contracts as the base class methods.
- When overriding methods, derived classes must respect the base class's method contracts. Specifically:
  - **Preconditions**: Derived class methods should require the same or weaker preconditions (e.g., input constraints) than the base class method. This ensures that the derived class does not impose stricter requirements that could break client expectations.
  - **Postconditions**: Derived class methods must provide the same or stronger postconditions (e.g., output guarantees) as the base class method, ensuring that the method's results meet or exceed the base class's promises.
  - **Invariants**: Derived classes must maintain all invariants defined by the base class, ensuring that the object's state remains valid according to the base class's rules.

- Use polymorphism to allow derived class objects to replace base class objects, often by overriding methods to provide specialized behavior while maintaining the core functionality of the base class.

Interface Segregation Principle (ISP)

The "I" in the SOLID acronym stands for the Interface Segregation Principle (ISP), which emphasizes that clients (classes or components that use interfaces) should not be forced to depend on interfaces they don't use. In other words, an interface should have a specific and focused set of methods that are relevant to the implementing classes.

Violation of ISP

Here's an example of a design that violates ISP by including methods that not all implementing classes need:

```
interface Worker {
    void work();

    void eat();

    void sleep();
}

class Robot implements Worker {
    public void work() {
        System.out.println("Performing tasks like welding.");
    }

    public void eat() {
        throw new UnsupportedOperationException("Robots don't eat.");
    }

    public void sleep() {
        throw new UnsupportedOperationException("Robots don't sleep.");
    }
}

class Human implements Worker {
    public void work() {
        System.out.println("Performing tasks like coding.");
    }

    public void eat() {
        System.out.println("Eating a meal.");
    }

    public void sleep() {
        System.out.println("Sleeping for rest.");
    }
}
```

In this code, the Worker interface forces Robot to implement eat and sleep, which are irrelevant, leading to unsupported operations. This makes the code harder to maintain and prone to errors, as classes must handle methods that don't apply to them.

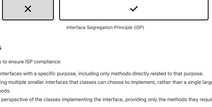
Fixing the violation

To align with ISP, we can refactor this interface to be more focused:

- Instead of a single Worker interface with unrelated methods, we split it into three smaller, tailored interfaces: Workable, Eatable, and Sleepable.
- The Workable interface includes only the work method, which all workers, like robots and humans, can implement.
- The Eatable interface includes the eat method, relevant to humans but not robots.
- The Sleepable interface includes the sleep method, which is also specific to humans.

This design ensures classes implement only the methods they need, making the code cleaner and more maintainable.

A visual representation of the refactored interfaces (Workable, Eatable, Sleepable) adhering to the ISP is shown below.



Interface Segregation Principle (ISP)

Best Practices

Here are guidelines to ensure ISP compliance:

- Aim to design interfaces with a specific purpose, including only methods directly related to that purpose.
- Consider creating multiple smaller interfaces that classes can choose to implement, rather than a single large interface with unrelated methods.
- Think from the perspective of the classes implementing the interface, providing only the methods they require.

Dependency Inversion Principle (DIP)

The Dependency Inversion Principle (DIP) states that high-level modules (or classes) should not depend on low-level modules; both should depend on abstractions, such as interfaces. In simpler terms, the principle encourages the use of abstract interfaces to decouple higher-level components from lower-level details.

Violation of DIP

Here's an example of a design that violates DIP by having a high-level class depend directly on a low-level class:

```
class LightBulb {
    public void turnOn() {
        System.out.println("LightBulb is on.");
    }

    public void turnOff() {
        System.out.println("LightBulb is off.");
    }
}

class Switch {
    private LightBulb bulb;

    public Switch(LightBulb bulb) {
        this.bulb = bulb;
    }

    public void operate() {
        bulb.turnOn();
    }
}
```

The Switch class above violates DIP because it depends directly on the low-level LightBulb class, rather than an abstraction. This tight coupling means that changing the LightBulb class, such as modifying its methods, or replacing it with another device, like a Fan, requires altering the Switch class's code. This makes the system less flexible and harder to maintain, as high-level modules should not be tied to low-level details.

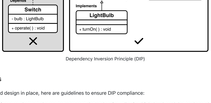
Fixing the violation

To align with DIP, we can refactor this design to use abstractions:

- We introduce a Switchable interface that defines standard methods, such as turnOn and turnOff, which any switchable device can implement.
- The Switch class is modified to depend only on the Switchable interface, not on specific devices.
- The LightBulb class implements the Switchable interface, providing its turnOn and turnOff behavior.

Now, Switch can work with any device that implements Switchable, like a Fan or a Heater, without needing changes to its code. This design decouples Switch from low-level details, making the system more flexible and easier to extend.

A visual representation of the dependency inversion is shown below.



Dependency Inversion Principle (DIP)

Best Practices

With this decoupled design in place, here are guidelines to ensure DIP compliance:

- Introduce interfaces or abstract classes to represent dependencies, allowing high-level modules to depend on these abstractions.
- Use dependency injection to inject concrete implementations into high-level modules through their abstractions. This promotes loose coupling.

Wrap Up

This chapter has helped you understand how to use Encapsulation, Abstraction, Inheritance, Polymorphism, and the SOLID principles. These concepts form the backbone of robust software design, enabling you to create flexible, maintainable, and scalable systems. Applying these tools will help you articulate clear design decisions, justify your approach, and demonstrate adherence to industry-standard practices.

To further refine your skills and deepen your expertise, explore the following resources:

- 'Clean Code: A Handbook of Agile Software Craftsmanship' by Robert C. Martin
- 'Design Patterns: Elements of Reusable Object-Oriented Software' by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- 'Tidy First?: A Personal Exercise in Empirical Software Design' by Kent Beck

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.